Introduction to Data Science — Final Exam Paper 7 (Extreme Difficult)

Course Code: BCSE2C05 | **Semester:** II | **Max Marks:** 100 | **Time:** 3 Hours **Difficulty:** ★★★★★ Extreme Difficult

---

SECTION A — Short Answer (5 Marks Each, Do Any 4 of 5 = 20 Marks)

Q1. Explain the curse of dimensionality. How does it affect k-NN and K-Means?

Curse of Dimensionality: As the number of features (dimensions) increases, the data becomes increasingly sparse, and distance-based measures become less meaningful.

Key Effects:

1. **Distance concentration:** In high dimensions, the difference between the nearest and farthest point approaches zero — all points appear equidistant.
2. **Volume explosion:** A unit hypercube in d dimensions needs exponentially more data to maintain the same density.
3. **Overfitting:** Models have more parameters than meaningful patterns.

Impact on k-NN:

- k-NN relies on distance to find neighbours. When all points are equidistant, the "nearest" neighbour is essentially random.
- Classification accuracy degrades significantly.
- **Solution:** Feature selection, PCA, or L1/L2 regularization to reduce dimensions.

Impact on K-Means:

- Cluster assignments become arbitrary when distances lose meaning.
- Centroids may not represent meaningful cluster centers.
- **Solution:** Apply PCA before clustering, use subspace clustering.

Q2. Given data, compute the Pearson Correlation Coefficient manually.

Data:

X	Y
1	2
2	4
3	5
4	4
5	5

Step 1: Means: $\bar{X} = 3$, $\bar{Y} = 4$

Step 2: Compute components:

X	Y	$X-\bar{X}$	$Y-\bar{Y}$	$(X-\bar{X})(Y-\bar{Y})$	$(X-\bar{X})^2$	$(Y-\bar{Y})^2$
1	2	−2	−2	4	4	4
2	4	−1	0	0	1	0
3	5	0	1	0	0	1
4	4	1	0	0	1	0
5	5	2	1	2	4	1

Σ				**6**	**10**	**6**

Step 3: $r = \Sigma(X-\bar{X})(Y-\bar{Y}) / \sqrt{[\Sigma(X-\bar{X})^2 \times \Sigma(Y-\bar{Y})^2]}$

$r = 6 / \sqrt{(10 \times 6)} = 6 / \sqrt{60} = 6 / 7.746 =$ **0.775**

Interpretation: Strong positive correlation — as X increases, Y tends to increase.

Q3. Write Python code to perform stratified train-test split and explain why it's important for imbalanced datasets.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

# Simulated imbalanced dataset
np.random.seed(42)
n = 1000
df = pd.DataFrame({
    'Feature1': np.random.randn(n),
    'Feature2': np.random.randn(n),
    'Target': np.concatenate([np.zeros(950), np.ones(50)])  # 95% vs 5%
})

print("Original class distribution:")
print(df['Target'].value_counts(normalize=True))

# ── Regular split (RISKY) ──
X = df[['Feature1', 'Feature2']]
y = df['Target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"\nRegular split — Train: {y_train.mean():.3f}, Test: {y_test.mean():.3f}")

# ── Stratified split (SAFE) ──
X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print(f"Stratified split — Train: {y_train_s.mean():.3f}, Test: {y_test_s.mean():.3f}")
```

Why Stratification Matters:

- Without stratification, the test set might have 0% or 15% of the minority class by chance — leading to unreliable evaluation.
- Stratification ensures both train and test sets have the **same class ratio** as the original dataset (5% positive in both).
- Critical for imbalanced datasets where random splits can create test sets with zero minority samples.

Q4. Explain binning as a data smoothing technique. Perform equal-frequency binning with smoothing by bin median.

Binning is a noise reduction technique that divides data into intervals (bins) and replaces values within each bin with a representative value.

Equal-Frequency (Equal-Depth) Binning: Each bin has the same number of data points.

Data (sorted): {4, 8, 9, 15, 21, 24, 25, 28, 34}

Number of bins = 3 → Each bin has 9/3 = 3 values.

Bin	Values	Bin Median	Smoothed by Median
Bin 1	4, 8, 9	8	8, 8, 8

Bin 2	15, 21, 24	21	21, 21, 21
Bin 3	25, 28, 34	28	28, 28, 28

Smoothed data: {8, 8, 8, 21, 21, 21, 28, 28, 28}

Other smoothing options:

- **By bin mean:** Replace each with average of bin (e.g., Bin 1: (4+8+9)/3=7).
- **By bin boundaries:** Replace each with nearest bin boundary (e.g., 4→4, 8→9, 9→9 for Bin 1 boundaries [4,9]).

Q5. Write a Python function to detect outliers using both IQR and Z-Score methods and return results as a DataFrame.

```
import pandas as pd
import numpy as np

def detect_outliers(df, column):
    """Detect outliers using both IQR and Z-Score methods."""
    data = df[column].dropna()

    # ── IQR Method ──
    Q1 = data.quantile(0.25)
    Q3 = data.quantile(0.75)
    IQR = Q3 - Q1
    iqr_lower = Q1 - 1.5 * IQR
    iqr_upper = Q3 + 1.5 * IQR
    iqr_outliers = (data < iqr_lower) | (data > iqr_upper)

    # ── Z-Score Method ──
    z_scores = (data - data.mean()) / data.std()
    z_outliers = np.abs(z_scores) > 3

    # ── Combine results ──
    results = pd.DataFrame({
        column: data,
        'Z_Score': z_scores.round(3),
        'IQR_Outlier': iqr_outliers,
        'ZScore_Outlier': z_outliers,
        'Both_Methods': iqr_outliers & z_outliers
    })

    print(f"── Outlier Report for '{column}' ──")
    print(f"IQR Bounds: [{iqr_lower:.2f}, {iqr_upper:.2f}]")
    print(f"IQR Outliers: {iqr_outliers.sum()}")
    print(f"Z-Score Outliers (|z|>3): {z_outliers.sum()}")
    print(f"Detected by both: {(iqr_outliers & z_outliers).sum()}")

    return results[results['IQR_Outlier'] | results['ZScore_Outlier']]

# Usage
df = pd.DataFrame({'Salary': [25000, 30000, 35000, 40000, 45000,
                              50000, 55000, 60000, 65000, 500000]})
outlier_report = detect_outliers(df, 'Salary')
print(outlier_report)
```

SECTION B — Analytical Questions (8 Marks Each, Do Any 5 of 6 = 40 Marks)

Q6. Critically analyse the trade-offs between model interpretability and accuracy. When should you choose a simpler model over a complex one?

The Interpretability-Accuracy Trade-off:

```
High Interpretability ◄──────────────────► High Accuracy
  Linear Regression    Decision Tree    Random Forest    Deep Learning
  Logistic Regression  k-NN             XGBoost          Neural Networks
```

Model	Interpretability	Accuracy	When to Use
Linear/Logistic Regression	★★★★★	★★☆☆☆	Regulated industries (finance, healthcare), feature importance needed
Decision Tree	★★★★☆	★★★☆☆	Explaining decisions to non-technical stakeholders
Random Forest	★★★☆☆	★★★★☆	Good balance of performance and partial interpretability
XGBoost/LightGBM	★★☆☆☆	★★★★★	Competitions, when accuracy is paramount
Deep Learning	★☆☆☆☆	★★★★★	Image/NLP tasks, when data is abundant

When to Choose Simpler Models:

1. **Regulatory compliance** — GDPR requires "right to explanation." Banks must justify loan denials.
2. **Medical diagnosis** — Doctors need to understand why a patient was flagged.
3. **Small datasets** — Complex models overfit; simpler models generalize better.
4. **Debugging** — Interpretable models are easier to troubleshoot.
5. **Stakeholder trust** — Business leaders trust models they can understand.

When Complexity is Justified:

1. **Image recognition** — Only deep learning achieves state-of-the-art.
2. **NLP tasks** — Transformer models are necessary for language understanding.
3. **When accuracy gap is significant** — 65% (linear) vs 95% (XGBoost) justifies complexity.

Explainability Tools for Complex Models:

- **LIME** — Local explanations for individual predictions.
- **SHAP** — Shapley values for global + local feature importance.
- **Feature Importance** — Built-in for tree-based models.

Q7. Design and implement a complete missing data analysis in Python that identifies the type of missingness (MCAR, MAR, MNAR) and applies appropriate techniques.

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# ─── Simulated dataset with different missing patterns ───
np.random.seed(42)
n = 500
data = {
    'Age': np.random.normal(35, 10, n),
    'Income': np.random.normal(60000, 20000, n),
    'Education_Years': np.random.randint(10, 22, n),
    'Satisfaction': np.random.choice([1,2,3,4,5], n)
}
df = pd.DataFrame(data)

# Introduce MCAR in Age (randomly)
```

```python
mcar_mask = np.random.random(n) < 0.1
df.loc[mcar_mask, 'Age'] = np.nan

# Introduce MAR in Income (missing when Education < 14)
mar_mask = df['Education_Years'] < 14
df.loc[mar_mask & (np.random.random(n) < 0.4), 'Income'] = np.nan

# Introduce MNAR in Satisfaction (low satisfaction people don't respond)
mnar_mask = df['Satisfaction'] <= 2
df.loc[mnar_mask & (np.random.random(n) < 0.5), 'Satisfaction'] = np.nan

# ── 1. Visualize Missingness ──
print("Missing Value Summary:")
print(df.isnull().sum())
print(f"\nMissing Percentages:\n{(df.isnull().mean() * 100).round(2)}%")

# Missingness heatmap
plt.figure(figsize=(8, 5))
sns.heatmap(df.isnull(), cbar=True, yticklabels=False, cmap='viridis')
plt.title('Missing Data Pattern')
plt.show()

# ── 2. Test for MCAR (Little's Test Proxy) ──
# If missingness in one column is independent of other columns → MCAR
# Compare means of observed groups
income_present = df[df['Income'].notna()]['Education_Years'].mean()
income_missing = df[df['Income'].isna()]['Education_Years'].mean()
print(f"\nMAR Test (Income):")
print(f"  Avg Education (Income present): {income_present:.2f}")
print(f"  Avg Education (Income missing): {income_missing:.2f}")
print(f"  Difference: {abs(income_present - income_missing):.2f}")
print("  → Significant difference suggests MAR (missingness depends on Education)")

# ── 3. Apply Appropriate Techniques ──

# MCAR (Age) → Safe to use any imputation
df['Age_imputed'] = df['Age'].fillna(df['Age'].median())

# MAR (Income) → Use conditional imputation
for edu_group in df['Education_Years'].unique():
    mask = (df['Education_Years'] == edu_group) & (df['Income'].isna())
    group_median = df[df['Education_Years'] == edu_group]['Income'].median()
    df.loc[mask, 'Income'] = group_median

# MNAR (Satisfaction) → Flag and impute conservatively
df['Satisfaction_missing'] = df['Satisfaction'].isna().astype(int)  # Keep as a feature
df['Satisfaction'] = df['Satisfaction'].fillna(df['Satisfaction'].median())

print("\nAfter Imputation — Missing Values:")
print(df.isnull().sum())
```

Types of Missingness:

Type	Meaning	Test	Imputation
MCAR	Completely random	Little's test	Any method (mean, median, listwise deletion)
MAR	Depends on observed variables	Compare group means	Conditional imputation, Multiple Imputation
MNAR	Depends on the missing value itself	Domain knowledge	Model-based, collect more data, flag as feature

---

Q8. Explain association rule mining concepts (Support, Confidence, Lift) with a numerical example. How is it applied in e-commerce?

Association Rule Mining discovers interesting relationships between variables in transactional data.

Key Metrics:

Metric	Formula	Meaning
Support	freq(A∪B) / N	How often A and B appear together
Confidence	freq(A∪B) / freq(A)	Probability of B given A
Lift	Confidence / Support(B)	How much more likely B is with A vs alone

Numerical Example:

Total transactions N = 1000

Itemset	Frequency
{Bread}	400
{Butter}	300
{Bread, Butter}	200

Rule: Bread → Butter

- Support = 200/1000 = **0.20** (20% of transactions have both)
- Confidence = 200/400 = **0.50** (50% of bread buyers also buy butter)
- Lift = 0.50 / (300/1000) = 0.50/0.30 = **1.67** (buying bread makes butter 1.67× more likely)

Interpretation:

- Lift > 1 → Positive association (products complement each other)
- Lift = 1 → Independent (no relationship)
- Lift < 1 → Negative association (products substitute each other)

E-Commerce Application (Amazon):

- Market basket analysis → "Frequently bought together" suggestions.
- Rule: {Phone} → {Phone Case, Screen Guard} (confidence=0.72, lift=3.5).
- Used for product placement, bundle pricing, cross-selling emails.
- Apriori algorithm finds frequent itemsets satisfying minimum support threshold, then generates rules above minimum confidence.

Q9. Write Python code for a complete data normalization pipeline comparing Min-Max, Z-Score, and Robust Scaling on a dataset with outliers.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Dataset with outliers
data = {
    'Feature': [10, 15, 20, 25, 30, 35, 40, 45, 50, 200]  # 200 is outlier
}
df = pd.DataFrame(data)

# ── 1. Min-Max Normalization ──
x = df['Feature']
df['MinMax'] = (x - x.min()) / (x.max() - x.min())
```

```
# ── 2. Z-Score Standardization ──
df['ZScore'] = (x - x.mean()) / x.std()

# ── 3. Robust Scaling (uses median and IQR – outlier resistant) ──
Q1 = x.quantile(0.25)
Q3 = x.quantile(0.75)
IQR = Q3 - Q1
df['Robust'] = (x - x.median()) / IQR

print("Comparative Normalization:")
print(df.round(3))

# ── Visualization ──
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

methods = ['MinMax', 'ZScore', 'Robust']
colors = ['steelblue', 'coral', 'seagreen']

for ax, method, color in zip(axes, methods, colors):
    ax.bar(range(len(df)), df[method], color=color, alpha=0.7)
    ax.set_title(f'{method} Scaling')
    ax.set_xlabel('Data Point Index')
    ax.set_ylabel('Scaled Value')
    ax.axhline(y=0, color='black', linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()
```

Comparison Table:

Aspect	Min-Max	Z-Score	Robust
Formula	(X–min)/(max–min)	(X–μ)/σ	(X–median)/IQR
Range	[0, 1]	~[–3, 3]	No fixed range
Outlier impact	× High (compresses normal data)	× Moderate (mean/std affected)	✅ Low (median/IQR robust)
Best for	Bounded, clean data	Normal distributions	Data with outliers

In our example, Min-Max compresses all non-outlier data into [0, 0.21] because 200 dominates. Robust scaling handles this correctly.

Q10. Write a Python program that implements K-Means clustering from scratch (without sklearn) and visualizes the clusters.

```
import numpy as np
import matplotlib.pyplot as plt

def kmeans(X, K, max_iters=100, tol=1e-4):
    """K-Means Clustering from scratch."""
    n_samples, n_features = X.shape

    # Step 1: Random initialization
    indices = np.random.choice(n_samples, K, replace=False)
    centroids = X[indices].copy()

    for iteration in range(max_iters):
        # Step 2: Assign clusters (nearest centroid)
        distances = np.zeros((n_samples, K))
        for k in range(K):
```

```
            distances[:, k] = np.sqrt(np.sum((X - centroids[k])**2, axis=1))
        labels = np.argmin(distances, axis=1)

        # Step 3: Update centroids
        new_centroids = np.zeros_like(centroids)
        for k in range(K):
            if np.sum(labels == k) > 0:
                new_centroids[k] = X[labels == k].mean(axis=0)
            else:
                new_centroids[k] = centroids[k]

        # Step 4: Check convergence
        shift = np.sqrt(np.sum((new_centroids - centroids)**2))
        print(f"Iteration {iteration+1}: Centroid shift = {shift:.6f}")

        if shift < tol:
            print(f"Converged at iteration {iteration+1}")
            break

        centroids = new_centroids

    return labels, centroids

# ── Generate 2D sample data ──
np.random.seed(42)
cluster1 = np.random.randn(50, 2) + [2, 2]
cluster2 = np.random.randn(50, 2) + [8, 3]
cluster3 = np.random.randn(50, 2) + [5, 8]
X = np.vstack([cluster1, cluster2, cluster3])

# ── Run K-Means ──
labels, centroids = kmeans(X, K=3)

# ── Visualize ──
colors = ['#e74c3c', '#3498db', '#2ecc71']
plt.figure(figsize=(8, 6))

for k in range(3):
    cluster_points = X[labels == k]
    plt.scatter(cluster_points[:, 0], cluster_points[:, 1],
                c=colors[k], label=f'Cluster {k+1}', alpha=0.6)

plt.scatter(centroids[:, 0], centroids[:, 1],
            c='black', marker='X', s=200, label='Centroids')
plt.title('K-Means Clustering (From Scratch)')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Algorithm Complexity: O(n × K × d × i) where n=points, K=clusters, d=dimensions, i=iterations.

Q11. Design a Spatial Visualization dashboard using Folium and Pandas that plots multiple markers from a CSV, adds popups, and uses marker clustering.

```
import folium
from folium.plugins import MarkerCluster
import pandas as pd
```

```
# ── Sample India Cities Data ──
data = {
    'City': ['Delhi', 'Mumbai', 'Bangalore', 'Chennai', 'Kolkata',
             'Hyderabad', 'Pune', 'Jaipur', 'Lucknow', 'Ahmedabad'],
    'Lat': [28.6139, 19.0760, 12.9716, 13.0827, 22.5726,
            17.3850, 18.5204, 26.9124, 26.8467, 23.0225],
    'Lon': [77.2090, 72.8777, 77.5946, 80.2707, 88.3639,
            78.4867, 73.8567, 75.7873, 80.9462, 72.5714],
    'Population_Lakhs': [190, 185, 85, 75, 45, 69, 50, 31, 28, 55],
    'AQI': [280, 160, 90, 110, 170, 100, 85, 200, 250, 140]
}
df = pd.DataFrame(data)

# ── Create Map ──
m = folium.Map(location=[22.0, 79.0], zoom_start=5,
               tiles='CartoDB dark_matter')

# ── Add Marker Cluster ──
marker_cluster = MarkerCluster().add_to(m)

for _, row in df.iterrows():
    # Color based on AQI
    if row['AQI'] < 100:
        color = 'green'
    elif row['AQI'] < 200:
        color = 'orange'
    else:
        color = 'red'

    # Popup with HTML
    popup_html = f"""
    <div style="font-family: Arial; width: 200px;">
        <h4>{row['City']}</h4>
        <p><b>Population:</b> {row['Population_Lakhs']} Lakhs</p>
        <p><b>AQI:</b> {row['AQI']}
           <span style="color:{color};">●</span></p>
    </div>
    """

    folium.Marker(
        location=[row['Lat'], row['Lon']],
        popup=folium.Popup(popup_html, max_width=250),
        tooltip=row['City'],
        icon=folium.Icon(color=color, icon='info-sign')
    ).add_to(marker_cluster)

# ── Add Circle Markers (proportional to population) ──
for _, row in df.iterrows():
    folium.CircleMarker(
        location=[row['Lat'], row['Lon']],
        radius=row['Population_Lakhs'] / 10,
        color='blue',
        fill=True,
        fill_opacity=0.3,
        tooltip=f"{row['City']}: {row['Population_Lakhs']}L"
    ).add_to(m)

# Save
```

```
m.save('india_cities_dashboard.html')
print("Dashboard saved as india_cities_dashboard.html")
```

SECTION C — Long Answer (20 Marks Each, Do Any 2 of 3 = 40 Marks)

Q12. (a) Compare all four types of ML with real-world case studies showing when each type fails (10 Marks). (b) Design a complete ML system for predicting hospital readmission within 30 days, addressing every stage from data to deployment (10 Marks).

(a) ML Types — When They Work and When They Fail:

Type	Works Best	Fails When	Case Study
Supervised	Large labelled data, clear input-output relationship	Labels are noisy, expensive, or unavailable	✅ Google's diabetic retinopathy detection (trained on 128K labelled images) → achieved specialist-level accuracy. × Fails when medical images are inconsistently labelled by different doctors.
Unsupervised	Exploratory analysis, no labels	Data has no clear cluster structure, high noise	✅ Spotify Discover Weekly (user clustering). × Fails when clusters overlap significantly or K is chosen poorly.
Semi-supervised	Few labels, abundant unlabelled data	Unlabelled data has very different distribution from labelled data	✅ GPT language models (few labelled + massive unlabelled text). × Fails when pseudo-labels are incorrect, causing error propagation.
Reinforcement	Sequential decision making, simulation available	Reward function is poorly defined, environment is too complex	✅ DeepMind's AlphaGo. × Fails in real-world robotics where a wrong action can cause physical damage (safe exploration problem).

Critical Insight: No single ML type is universally best — the choice depends on data availability, task structure, and consequences of errors.

(b) Hospital Readmission Prediction System:

Problem: Predict if a patient will be readmitted within 30 days of discharge.

Step 1 — Data Collection:

- EHR (Electronic Health Records): diagnosis codes (ICD-10), procedures, lab results.
- Patient demographics: age, gender, insurance type.
- Previous admissions: frequency, duration, departments visited.
- Medications: number of drugs, high-risk medications.
- Discharge notes: NLP extraction of key phrases.

Step 2 — Feature Engineering:

```
# Engineered features
df['num_diagnoses'] = df['diagnosis_codes'].str.count(',') + 1
df['length_of_stay'] = (df['discharge_date'] - df['admit_date']).dt.days
df['prior_admissions_6mo'] = df.groupby('patient_id')['admit_date'].transform(
    lambda x: x.rolling('180D').count() - 1
)
df['medication_count'] = df['medications'].str.count(';') + 1
df['has_diabetes'] = df['diagnosis_codes'].str.contains('E11').astype(int)
df['weekend_discharge'] = df['discharge_date'].dt.weekday.isin([5,6]).astype(int)
```

Step 3 — Handling Challenges:

- **Class imbalance** (~15% readmission rate): SMOTE + class weights.
- **Missing data:** Lab results often missing → flag missingness as a feature + median imputation.
- **High cardinality:** ICD-10 codes (68,000+) → group into CCS categories (285 groups).

Step 4 — Model Selection:

- Baseline: Logistic Regression (LACE index recreation).
- Primary: XGBoost with `scale_pos_weight` for imbalance.
- Ensemble: Stack LR + RF + XGBoost with meta-learner.

Step 5 — Evaluation:

Metric	Target	Rationale
AUC-ROC	> 0.75	Industry benchmark for readmission
Recall	> 0.70	Must catch high-risk patients
Precision	> 0.30	Acceptable false alarm rate
Calibration	Brier < 0.15	Predicted probabilities must be reliable

Step 6 — Deployment:

- Integrated into hospital EHR as a risk score (0-100) at discharge.
- Patients with score > 60 → automatic follow-up call scheduling.
- Monthly retraining with new discharge data.
- Monitor for data drift (seasonal illness patterns).

Q13. Write an advanced Python program that performs end-to-end EDA including multivariate analysis, advanced visualizations (pair plots, heatmaps, violin plots), and automated report generation.

Dataset: `housing_data.csv` with columns: `ID, Area_SqFt, Bedrooms, Bathrooms, Age_Years, Distance_City_KM, Has_Parking, Has_Garden, Price_Lakhs`

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# ═══════════════════════════════════════
# 1. Data Loading & Quality Audit
# ═══════════════════════════════════════
df = pd.read_csv('housing_data.csv')
print("=" * 50)
print("DATA QUALITY AUDIT")
print("=" * 50)
print(f"Shape: {df.shape}")
print(f"\nColumn Types:\n{df.dtypes}")
print(f"\nMissing Values:\n{df.isnull().sum()}")
print(f"\nDuplicate Rows: {df.duplicated().sum()}")
print(f"\nUnique Values:\n{df.nunique()}")

# ═══════════════════════════════════════
# 2. Statistical Summary
# ═══════════════════════════════════════
print("\n=" * 50)
print("STATISTICAL SUMMARY")
print("=" * 50)

numeric_cols = ['Area_SqFt', 'Bedrooms', 'Bathrooms',
                'Age_Years', 'Distance_City_KM', 'Price_Lakhs']

stats_report = pd.DataFrame({
    'Mean': df[numeric_cols].mean(),
```

```
    'Median': df[numeric_cols].median(),
    'Std': df[numeric_cols].std(),
    'Skewness': df[numeric_cols].skew(),
    'Kurtosis': df[numeric_cols].kurt(),
    'Q1': df[numeric_cols].quantile(0.25),
    'Q3': df[numeric_cols].quantile(0.75),
    'IQR': df[numeric_cols].quantile(0.75) - df[numeric_cols].quantile(0.25)
}).round(3)
print(stats_report)

# ═══════════════════════════════════════
# 3. Missing Data Handling
# ═══════════════════════════════════════
for col in numeric_cols:
    if df[col].isnull().sum() > 0:
        skewness = df[col].skew()
        if abs(skewness) > 1:
            # Skewed → use median
            df[col] = df[col].fillna(df[col].median())
            print(f"Filled {col} with MEDIAN (skewness={skewness:.2f})")
        else:
            # Normal → use mean
            df[col] = df[col].fillna(df[col].mean())
            print(f"Filled {col} with MEAN (skewness={skewness:.2f})")

# ═══════════════════════════════════════
# 4. Outlier Detection & Treatment
# ═══════════════════════════════════════
print("\n=" * 50)
print("OUTLIER DETECTION (IQR)")
print("=" * 50)

for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    count = ((df[col] < lower) | (df[col] > upper)).sum()
    print(f"{col}: [{lower:.0f}, {upper:.0f}] → {count} outliers")
    # Cap outliers
    df[col] = df[col].clip(lower=lower, upper=upper)

# ═══════════════════════════════════════
# 5. Correlation Analysis
# ═══════════════════════════════════════
correlation = df[numeric_cols].corr()

plt.figure(figsize=(10, 8))
mask = np.triu(np.ones_like(correlation, dtype=bool))
sns.heatmap(correlation, mask=mask, annot=True, fmt='.2f',
            cmap='RdBu_r', center=0, square=True,
            linewidths=0.5, cbar_kws={"shrink": 0.8})
plt.title('Feature Correlation Matrix (Lower Triangle)', fontsize=14)
plt.tight_layout()
plt.show()

# Top correlations with Price
price_corr = correlation['Price_Lakhs'].drop('Price_Lakhs').sort_values(ascending=False)
print("\nCorrelations with Price:")
for feature, corr in price_corr.items():
    strength = 'Strong' if abs(corr) > 0.6 else 'Moderate' if abs(corr) > 0.3 else 'Weak'
```

```
    print(f"  {feature}: {corr:.3f} ({strength})")

# ═══════════════════════════════════════
# 6. Advanced Visualizations
# ═══════════════════════════════════════

# 6a. Pair Plot
sns.pairplot(df[['Area_SqFt', 'Bedrooms', 'Price_Lakhs', 'Has_Parking']],
             hue='Has_Parking', palette='Set1', diag_kind='kde')
plt.suptitle('Pair Plot — Key Features', y=1.02)
plt.show()

# 6b. Multi-panel Dashboard
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('Housing Data EDA Dashboard', fontsize=16, fontweight='bold')

# Histogram with KDE
sns.histplot(df['Price_Lakhs'], kde=True, color='steelblue', ax=axes[0,0])
axes[0,0].set_title('Price Distribution')

# Violin Plot
sns.violinplot(x='Has_Parking', y='Price_Lakhs', data=df,
               palette='Set2', ax=axes[0,1])
axes[0,1].set_title('Price by Parking')

# Boxplot
sns.boxplot(x='Bedrooms', y='Price_Lakhs', data=df,
            palette='pastel', ax=axes[0,2])
axes[0,2].set_title('Price by Bedrooms')

# Scatter with regression
sns.regplot(x='Area_SqFt', y='Price_Lakhs', data=df,
            scatter_kws={'alpha':0.4}, ax=axes[1,0])
axes[1,0].set_title('Area vs Price (with trend)')

# Count plot
sns.countplot(x='Bedrooms', data=df, palette='viridis', ax=axes[1,1])
axes[1,1].set_title('Bedroom Count Distribution')

# KDE plot
sns.kdeplot(data=df, x='Distance_City_KM', hue='Has_Garden',
            fill=True, ax=axes[1,2])
axes[1,2].set_title('Distance by Garden')

plt.tight_layout()
plt.show()

# ═══════════════════════════════════════
# 7. Statistical Test
# ═══════════════════════════════════════
print("\n=" * 50)
print("HYPOTHESIS TESTING")
print("=" * 50)

# Do houses with parking cost more?
with_parking = df[df['Has_Parking'] == 1]['Price_Lakhs']
without_parking = df[df['Has_Parking'] == 0]['Price_Lakhs']

t_stat, p_val = stats.ttest_ind(with_parking, without_parking)
print(f"\nT-Test: Parking vs Price")
print(f"  Mean with parking: ₹{with_parking.mean():.2f}L")
```

```
print(f"  Mean without: ₹{without_parking.mean():.2f}L")
print(f"  t-stat: {t_stat:.4f}, p-value: {p_val:.6f}")
print(f"  {'SIGNIFICANT' if p_val < 0.05 else 'NOT SIGNIFICANT'} at α=0.05")

# ═══════════════════════════════════════
# 8. Feature Engineering
# ═══════════════════════════════════════
df['Price_per_SqFt'] = df['Price_Lakhs'] * 100000 / df['Area_SqFt']
df['Total_Rooms'] = df['Bedrooms'] + df['Bathrooms']
df['Age_Category'] = pd.cut(df['Age_Years'],
    bins=[0, 5, 15, 30, 100],
    labels=['New', 'Recent', 'Old', 'Very_Old'])
df['Location_Score'] = 100 - df['Distance_City_KM']  # Closer is better

# ═══════════════════════════════════════
# 9. Save Processed Data
# ═══════════════════════════════════════
df.to_csv('housing_processed.csv', index=False)
print("\n✅ Processed data saved to housing_processed.csv")

# ═══════════════════════════════════════
# 10. Auto-Generated Report Summary
# ═══════════════════════════════════════
report = f"""
═══════════════════════════════════
  EDA REPORT — Housing Data
═══════════════════════════════════
Records: {df.shape[0]} | Features: {df.shape[1]}
Missing: {df.isnull().sum().sum()} (after imputation)

Top Price Predictors:
{price_corr.head(3).to_string()}

Price Statistics:
  Mean: ₹{df['Price_Lakhs'].mean():.2f} Lakhs
  Median: ₹{df['Price_Lakhs'].median():.2f} Lakhs
  Std Dev: ₹{df['Price_Lakhs'].std():.2f} Lakhs

Key Finding: Houses with parking cost {'more' if with_parking.mean() > without_parking.mean() else 'less'}
(p-value: {p_val:.4f}, {'statistically significant' if p_val < 0.05 else 'not significant'})
═══════════════════════════════════
"""
print(report)
```

Q14. (a) Critically discuss how the 5 V's of Big Data create specific technical and ethical challenges for Data Science practitioners, with examples from Indian context (10 Marks). (b) Design a comprehensive data governance framework for a large organization processing both structured and unstructured data (10 Marks).

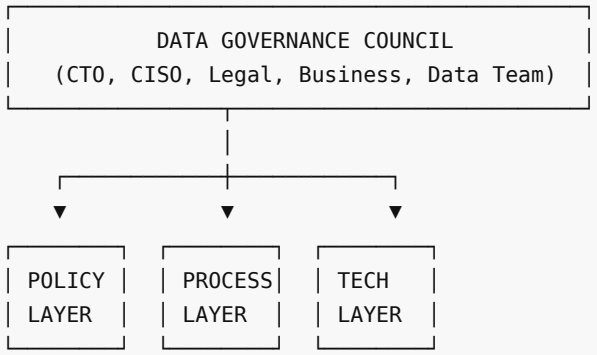
(a) 5 V's — Technical and Ethical Challenges (Indian Context):

V	Technical Challenge	Ethical Challenge	Indian Example
Volume	Storage costs, processing time, need for distributed systems (Hadoop/Spark).	Retaining massive personal data increases breach risk.	Aadhaar database (1.3B+ records) — world's largest biometric DB. Storage costs ₹500+ crores. Single breach exposes entire population.
Velocity	Real-time processing requires streaming infrastructure (Kafka, Flink).	Real-time surveillance enables mass monitoring.	UPI transactions (10B+/month) process in < 2 seconds. But real-time access to financial data raises surveillance concerns.

Variety	Integrating structured (databases), semi-structured (JSON), unstructured (video, text) requires different tools.	Combining data types can create detailed profiles without consent.	DigiLocker + CoWIN + Aarogya Setu — combining health records, ID documents, and location creates comprehensive citizen profiles.
Veracity	Noisy, inaccurate data leads to wrong insights. How to validate data from unreliable sources?	Decisions based on low-quality data can be discriminatory.	Election misinformation on WhatsApp — fake data spread widely, poisoning sentiment analysis models used by media.
Value	Extracting value requires expensive talent and infrastructure.	Value extraction can exploit users whose data creates that value.	Swiggy/Zomato use delivery partner GPS data to optimize operations — but delivery partners don't benefit from the insights generated from their data.

Cross-cutting Challenge — Digital Divide: India's digital divide means Big Data disproportionately represents urban, smartphone-using populations — rural and elderly communities are underrepresented, leading to biased models.

(b) Data Governance Framework:



1. Policy Layer:

Policy	Description
Data Classification	Public / Internal / Confidential / Restricted.
Retention Policy	Define how long each data type is kept (e.g., transaction data: 7 years, logs: 90 days).
Access Policy	Role-based access control (RBAC). Least privilege principle.
Privacy Policy	GDPR/DPDP Act compliance. Purpose limitation.
Ethical Use Policy	Prohibited uses, algorithmic fairness standards, bias audit requirements.

2. Process Layer:

Process	Description
Data Cataloging	Metadata repository documenting all datasets, owners, lineage, quality scores.
Data Quality Monitoring	Automated checks: completeness (>95%), accuracy, consistency, timeliness.
Change Management	Schema changes require approval; backward compatibility maintained.
Incident Response	Breach notification within 72 hours (GDPR). Root cause analysis.
Audit Trail	Log all data access, modifications, and exports.

3. Technology Layer:

Component	Tools/Implementation
Data Lake	Store raw structured + unstructured data (S3, HDFS).

Data Warehouse	Processed, query-ready structured data (Snowflake, BigQuery).
Encryption	AES-256 at rest, TLS 1.3 in transit, column-level encryption for PII.
Anonymization	k-Anonymity, l-Diversity, differential privacy for analytics.
Monitoring	Data drift detection, quality dashboards, anomaly alerts.
Version Control	DVC (Data Version Control) for dataset versioning.

4. People & Culture:

Element	Implementation
Data Stewards	Assigned per department, responsible for data quality.
Training	Quarterly data literacy and ethics training for all employees.
Data Ethics Board	Reviews high-risk AI applications before deployment.

5. Compliance (India-specific):

- **DPDP Act 2023** — Personal Data Protection.
- **IT Act 2000** — Electronic data regulations.
- **RBI Guidelines** — Financial data localization (data must stay in India).
- **SEBI** — Securities data handling for trading firms.

— End of Paper 7 (Extreme Difficult) —